

Day 25 : JS Code Execution

The Fundamental Problem: Computers are Not Smart

The most important thing to remember is that a computer's processor (CPU) doesn't understand JavaScript. It only understands one thing: **Machine Code** (sequences of 1s and 0s).

Our goal is to translate the human-friendly JavaScript we write into machine-friendly instructions. This translation job is done by the **JavaScript Engine**.

The JavaScript Engine: A program that reads, translates, and executes JavaScript code. The most famous one is Google's **V8**, which powers Chrome and Node.js.

The Engine's Two Core Components

To manage our code, the engine sets up two primary areas in memory:

1. **The Memory Heap:** A large, unstructured pool of memory. Think of it as a giant warehouse where all the "things" (objects, arrays, and most importantly, functions) from your code are stored.
2. **The Call Stack:** A highly organized, temporary workspace. Think of it as a to-do list where you can only add tasks to the top and can only work on the topmost task. It's responsible for managing which function is currently running.

The Two-Phase Process: The Secret to "Hoisting"

The engine does not just read your code from top to bottom and run it. For any piece of code (the global script, a function), it performs a two-phase process.

1. **Creation Phase (Memory Setup):** The engine takes a quick first pass through the code. It doesn't execute anything. Its only job is to find all the variable and function declarations and set aside memory for them.
2. **Execution Phase (Doing the Work):** After the setup is complete, the engine takes a second pass to actually execute the code: assigning values to

variables, calling functions, and running the logic.

This two-phase process is what creates the effect we call **hoisting**.

A Detailed Walkthrough: From Code to Console

Let's trace the following code step-by-step to see every part of the process.

Our Code Example:

```
// script.js
var score = 50;
const playerName = "Alex";

function calculateBonus(currentScore) {
  var bonus = currentScore / 10;
  return bonus;
}

var finalScore = score + calculateBonus(score);
console.log(finalScore);
```

Step 0: The "Before You Run" Phase (Compilation)

Before anything else, the V8 engine takes our `script.js` file and compiles it.

1. **Parsing:** The code is read and turned into a data structure called an **Abstract Syntax Tree (AST)**.
2. **Compilation:** The AST is then compiled into **Bytecode**. This is a low-level set of instructions that the engine can execute very quickly. This bytecode is loaded into a special, executable area of memory called the **Code Space**.

The engine will not be reading our text file during execution; it will be running this highly optimized bytecode.

Step 1: Global Execution - The Creation Phase

The engine is now ready to run the global part of our script. It starts with the creation phase.

1. A **Global Execution Context (GEC)** is created. This is the main environment for our script. It is pushed onto the **Call Stack**.
2. The engine scans the global code for declarations:
 - `var score;` : Finds this. In the GEC's memory, it creates a property `score` and initializes it with `undefined`.
 - `const playerName;` : Finds this. It creates a `playerName` binding but leaves it in an **uninitialized** state. This is the start of its **Temporal Dead Zone (TDZ)**.
 - `function calculateBonus(...)` : Finds this. This is special.
 - A complete **Function Object** is created on the **Memory Heap**. This object contains the function's metadata and, crucially, a pointer to its executable bytecode in the Code Space.
 - In the GEC's memory, a property `calculateBonus` is created and is immediately initialized with a pointer to that object on the Heap.
 - `var finalScore;` : Finds this. Creates a `finalScore` property and initializes it with `undefined`.

Memory State After Global Creation:

- **Call Stack:** `[ GEC ]`
- **Global Memory:** `score: undefined` , `playerName: <uninitialized>` , `calculateBonus: <pointer to Heap>` , `finalScore: undefined`
- **Heap:** Contains the `calculateBonus` Function Object.

Step 2: Global Execution - The Execution Phase

The engine now executes the global bytecode instructions.

- `var score = 50;`
 - The value `50` is assigned to `score` in global memory.
- `const playerName = "Alex";`
 - The value `"Alex"` is assigned to `playerName`. Its TDZ is now over.
- `function calculateBonus(...)`

- This is a declaration. It was fully handled in the creation phase, so the engine skips it.
- `var finalScore = score + calculateBonus(score);`
 - This is the most important line. The engine needs to resolve this expression from right to left.
 - It first needs to execute `calculateBonus(score)`.
 - It looks up `score` in global memory and gets its value, `50`.
 - It prepares to call `calculateBonus` with the argument `50`. **The execution of the global code is now paused.**

Step 3: The Function Call - `calculateBonus(50)`

This is where the Call Stack comes into play.

1. **A new Function Execution Context (FEC)** is created for this specific call.
2. Before jumping to the function's code, the engine creates a **"bookmark"**. It takes the memory address of the **next bytecode instruction** needed to complete the `var finalScore = ...` line and stores this **Return Address** inside the new FEC.
3. The FEC is **pushed onto the top of the Call Stack**.
4. **FEC Creation Phase:**
 - The engine creates a local environment for this function.
 - The parameter `currentScore` is created as a local variable and initialized with the passed value, `50`.
 - The engine scans the function's code for declarations: it finds `var bonus;`. A local variable `bonus` is created and initialized with `undefined`.
5. **FEC Execution Phase:** The engine now executes the function's bytecode.
 - `var bonus = currentScore / 10;` : It looks up `currentScore` (finds `50` locally), calculates `50 / 10 = 5`, and assigns `5` to the local variable `bonus`.
 - `return bonus;` : The function needs to return. It looks up the value of `bonus` (which is `5`).

Visualizing the Call Stack at its Deepest Point:

```
TOP
+-----+
| calculateBonus() Exec. Context | ← We are executing here
| - Return Address: <to line 8> |
| - Local Memory:              |
| - currentScore: 50           |
| - bonus: 5                   |
+-----+
| Global Execution Context      | ← This is paused
| - Global Memory:             |
| - score: 50                  |
| - ...                        |
+-----+
```

Step 4: The Return

1. The `calculateBonus` function is finished. It packages up the return value, `5`.
2. The engine looks at the current FEC, finds the **Return Address** ("the bookmark"), and prepares to jump back.
3. The **FEC is popped from the Call Stack**. All of its local variables (`currentScore` , `bonus`) are instantly destroyed. The memory is cleaned up.
4. The engine's Program Counter is set to the retrieved Return Address.

Step 5: Resuming Global Execution

- Execution is now back on the line `var finalScore = ...`.
- The expression has been resolved to `50 + 5`.
- The engine calculates `55` and assigns this value to `finalScore` in global memory.
- `console.log(finalScore);`
 - This is another function call. A new FEC for `console.log` is pushed to the stack.

- It looks up `finalScore` (gets `55`), performs its job of printing to the console.
- The `console.log` FEC is popped from the stack.

The script is now finished. The GEC is popped from the Call Stack, and the program ends.

Key Takeaways

1. **Code is Compiled First:** Your JavaScript text is transformed into optimized Bytecode before it runs.
2. **Creation Before Execution:** Memory is set aside for variables and functions *before* the code is executed. This is why function declarations can be called before they appear in the code.
3. **The Call Stack Manages Flow:** The Call Stack keeps track of which function is currently running. Every function call creates a new, temporary, isolated environment (an Execution Context).
4. **The Stack is for "Who" and "Where":** It holds the execution contexts and the "return addresses" (bookmarks) to manage the flow of the program.
5. **The Heap is for "What":** It holds the actual objects and functions, which can live on long after the function that created them has finished.

Hoisting

The Misleading Idea (The Lie)

The common explanation is: "Hoisting is when JavaScript moves your variable and function declarations to the top of their scope before execution."

This is a lie. Nothing is physically moved. Forgetting this lie is the first step to truly understanding.

The Real Mechanism (The Truth)

The truth is that JavaScript code runs in two phases (or passes):

1. **Creation Phase (Memory Setup):** Before executing any code, the JavaScript engine reads through your code and sets aside memory for all the variables

and functions it finds. This is like a chef doing *mise en place*—getting all the ingredients ready before starting to cook.

2. **Execution Phase (Doing the Work):** After setting up memory, the engine goes back through the code line by line and actually executes it (assigns values, calls functions, etc.).

Hoisting is simply the visible effect of this two-phase process. Because the engine already knows about your variables and functions from the first pass, it feels like they were "hoisted" to the top.

The Unforgettable Analogy: The Teacher and the Roster

Imagine a teacher (the JavaScript Engine) walking into a classroom (your code's scope).

The teacher does two things:

1. **Takes Attendance (Creation Phase):** Before starting the lesson, the teacher takes out the class roster and reads all the students' names. They now *know which students are supposed to be in the class*.
2. **Teaches the Lesson (Execution Phase):** The teacher starts teaching from the first page of the textbook to the last.

Now, let's see how `var`, `let`, `const`, and `function` behave like different types of students during the **"Attendance" phase**.

1. Function Declarations: The Eager Student

```
// You ask the teacher about the student before the lesson starts
console.log(sayHello()); // "Hello!"

function sayHello() {
  return "Hello!";
}
```

- **Attendance (Creation Phase):** The teacher sees `function sayHello` on the roster. This student is so eager, they are **already in their seat and have their homework done**. The teacher marks them as "Present and Ready."

- **Result:** You can ask about this student (`call the function`) even before the lesson gets to their page, because the teacher already knows everything about them.

2. `var` Declarations: The Absent Student with a Reserved Desk

```
// You ask about the student's homework
console.log(studentName); // undefined

var studentName = "Alice";

console.log(studentName); // "Alice"
```

- **Attendance (Creation Phase):** The teacher sees `var studentName` on the roster. The teacher knows this student exists, so they reserve a desk for them, but the student isn't actually here yet. The teacher marks their status as "Absent" (which in JavaScript is `undefined`).
- **Result:** If you ask about `studentName` before the lesson reaches their line, the teacher will tell you their status: "Absent" (`undefined`). They won't say "I don't know who that is" (`ReferenceError`). They know the student exists, but they have no value yet.

3. `let` and `const` Declarations: The Late Student (Strict Rule)

```
// You ask about the student
console.log(score); // Uncaught ReferenceError: Cannot access 'score' before
initialization

let score = 100;
```

- **Attendance (Creation Phase):** The teacher sees `let score` on the roster. The teacher knows this student is supposed to be in the class, but has a very strict rule: **"No one is allowed to talk about this student or ask for their homework until they physically walk through the classroom door."** This period of time before they arrive is the **Temporal Dead Zone (TDZ)**.

- **Result:** If you ask about `score` before the lesson reaches its line, the teacher (JS Engine) doesn't just say they're absent. They stop the entire class and give you a `ReferenceError`. You broke the strict rule.
-

The Final, Memorable Definition

Forget the "moving code" idea. Remember this instead:

Hoisting is JavaScript's behavior of knowing about a variable or function's existence before executing the code. How it treats that knowledge depends on the keyword (function, var, let, or const).

- `function` : Hoisted completely (name and body).
- `var` : Hoisted and initialized with `undefined`.
- `let` / `const` : Hoisted, but not initialized. They are put in a Temporal Dead Zone.

```

1  var a = 10;
2  var b = 20;
3
4  function addNumber(num1,num2){
5      var sum = num1+num2;
6      return sum;
7  }
8
9  var sumResult1 = addNumber(a,b);
10 var sumResult2 = addNumber(4,5);
11 console.log(sumResult1,sumResult2);

```

Code Run --> Execution Context

1: Memory allocation
2: Execution Phase

Memory

Execution

Memo Execution

num1 = 10	
num2 = 20	
sum = 30	

a = 10	
b = 20	
addNumber = {fn code}	
sumResult1 = 30	
sumResult2 = 9	

num1 = 4	
num2 = 5	
sum = 9	

GEC: Global Execution Context

Byte Code
Code space

```

1  var a = 10;
2  var b = 20;
3
4  function addNumber(num1,num2){
5      var sum = num1+num2;
6      return sum;
7  }
8
9  var sumResult1 = addNumber(a,b);
10 var sumResult2 = addNumber(4,5);
11 console.log(sumResult1,sumResult2);

```

Stack

Heap

